GLD / GOLD

Create and use a Hyperlane Warp Route on Windows

BSC testnet and Ethereum Sepolia

A detailed PowerShell guide with every operational command, expected results, recovery steps, complete helper source listings, and a worked deployment.

Two paths: use the deployed route, or create a separate fresh route. Your existing GLD route has already passed a real 1 GLD round trip. Do not redeploy it just to move from Mac to Windows.

Network	Chain / domain	Gas currency
BSC testnet	97	Native test BNB
Ethereum Sepolia	11155111	Sepolia ETH

Version focus: Hyperlane CLI 45.0.1 • ethers 6.17.0 • Solidity compiler 0.8.38

Prepared 29 October 2026. On-chain workflow tested on macOS; Windows command syntax and launcher adapted for this guide. Native Windows execution was not available for validation.

Read this before starting

The goal is one token supply that can move between two chains. Deploy Gold (GLD) on BSC testnet. Its collateral router locks the real test GLD and messages Sepolia, where a synthetic GLD router mints the same quantity. Returning tokens burns the Sepolia representation and unlocks BSC collateral.

What this guide delivers

Gold has 18 decimals and a fixed initial supply of 1,000,000 GLD. The standard token contract has no public mint function. This is a test asset; it has no physical gold backing or redemption mechanism.

Choose your path

Existing route: install tools, extract existing-route, configure the owner key, run checks, then use the transfer steps. Skip fresh configuration and both deployment steps.

Fresh route: extract fresh-route, set the owner wallet, configure, fund, compile, deploy the token, deploy the route, and test both directions. New deployments have new addresses.

> The companion GLD-Windows-Kit.zip contains both paths and all files used by npm scripts. Commands such as npm.cmd run deploy:token are project scripts, not commands built into Hyperlane.

Read order

02 tools → 03 files → 04 wallet → 05 gas → 06 settings → 07 fresh config → 08 preflight → 09 token → 10 route → 11 verify → 12 outbound → 13 return → 14 retry → 15 troubleshooting → 16 backups → 17 worked example → 18 references. Full script listings follow.

Install Windows tools

Use Windows 10 or 11 and open Windows PowerShell from Start. Run one code block at a time. The guide works with Windows PowerShell 5.1 syntax; no WSL, Docker, Git, Foundry, or global Hyperlane installation is required.

Install Node.js

```
winget --version
winget install --id OpenJS.NodeJS.LTS --exact --source winget
```

WinGet may ask you to accept installer/source terms or approve installation. After installation, close PowerShell and open a new window so PATH is refreshed. If WinGet is missing, install App Installer from the Microsoft Store or use the official Node.js Windows installer [1, 2].

```
node --version
npm.cmd --version
where.exe node
where.exe npm
```

Use a maintained Node.js LTS release that satisfies the project requirements. This guide needs Node >=22.9 because the scripts use --env-file-if-exists; CLI 45.0.1 itself declares Node >=22. A current LTS release is suitable, but the exact Windows combination must be checked locally.

Why commands use npm.cmd

PowerShell can select npm.ps1, which may be blocked by the system execution policy. Explicitly running npm.cmd uses the standard Windows launcher without changing that policy.

```
npm.cmd --version
```

> Do not globally disable PowerShell execution-policy protections to fix an npm.ps1 error. Use npm.cmd throughout this guide. Do not type the PS C:\...> prompt when copying commands.

Extract and open the project

Download GLD-Windows-Kit.zip from this chat and save it to your Downloads folder using that exact filename. These commands extract it into a dedicated folder under your Windows user profile.

```
$Zip = Join-Path $env:USERPROFILE 'Downloads\GLD-Windows-Kit.zip'
$Lab = Join-Path $env:USERPROFILE 'gld-hyperlane-windows'
Expand-Archive -LiteralPath $Zip -DestinationPath $Lab
$Kit = Join-Path $Lab 'GLD-Windows-Kit'
Get-ChildItem $Kit
```

If the destination already exists, inspect it instead of using -Force and overwriting existing work. If the browser renamed the ZIP, adjust $Zip to its actual filename.

Select ONE folder

```
# Existing route: operate the deployed contracts
Set-Location (Join-Path $Kit 'existing-route')

# Fresh route: create separate contracts
Set-Location (Join-Path $Kit 'fresh-route')
```

Install pinned dependencies

```
Get-Location
Test-Path .\package.json
npm.cmd ci
npm.cmd ls --depth=0
npm.cmd run hyperlane -- --version
```

Expect CLI 45.0.1 and the dependency versions listed in package.json. The kit includes package-lock.json; npm ci installs that lockfile. It does not include node_modules or credentials.

> All remaining commands assume PowerShell is in the selected project folder. npm.cmd run status is a kit helper that loads .env and invokes Hyperlane status with the local registry.

Prepare the owner wallet

For a fresh route, use a dedicated development wallet that has no real assets. For the existing deployed route, use the owner account below. The token deployment script requires the signing account and OWNER_ADDRESS to be the same.

```
0x3ae38EDD6eDfD90180EC897cbF0C5BA527459C83
```

Public address versus private key

OWNER_ADDRESS is the public 0x address used by faucets and explorers. HYP_KEY is the private key for that account. A private key normally has 64 hexadecimal characters, optionally prefixed by 0x. A 12/24-word recovery phrase is different and must not be entered into HYP_KEY.

In your wallet, select the exact account, open its account-details/private-key export option, authenticate locally, and copy only that account private key. Wallet UI labels vary. Hardware-wallet accounts may not allow export; this kit uses a local software key, so use a separate funded development account for a fresh lab.

Create and edit the local environment file

```
if (-not (Test-Path .env)) {
    Copy-Item .env.example .env
}
notepad.exe .env
```

In Notepad, fill the existing HYP_KEY line and OWNER_ADDRESS line. Save the file exactly as .env, not .env.txt. Do not put a real key in a command, worksheet, screenshot, Git repository, or PDF.

```
HYP_KEY=0xYOUR_ACCOUNT_PRIVATE_KEY_HERE
OWNER_ADDRESS=0xYOUR_PUBLIC_WALLET_ADDRESS_HERE
GLD_SUPPLY=1000000
```

> The same account key controls that address on other EVM networks, including mainnet. Use the account private key only. The kit contains no recovery phrase and does not need one.

Fund the correct gas assets

You need native test BNB on BSC testnet and native ETH on Ethereum Sepolia. A LINK, USDT, wrapped BNB, or other ERC-20 balance does not pay native deployment gas.

Get test BNB

The official BNB faucet can impose a mainnet-balance eligibility rule. The BNB documentation also lists official support channels for test-fund requests [3]. Another option used successfully in this project was the GHOST BNB testnet faucet [4], which sent 0.01 test BNB. Availability and limits can change.

Enter only your public owner address into the faucet, select BNB Testnet / chain 97, and complete any human verification yourself. Never provide the private key or recovery phrase to a faucet. You do not have to purchase mainnet BNB if a free support or alternate-faucet route is available.

Get Sepolia ETH

Use an Ethereum Sepolia faucet and request funds for the same public address. Verify that the wallet is on Ethereum Sepolia (chain 11155111), not BSC testnet or Ethereum mainnet. Faucet eligibility varies.

Funding used in our worked example

```
BSC testnet:       0.01 native test BNB
Ethereum Sepolia:  0.05 Sepolia ETH
```

These are observed starting balances for this exercise, not guaranteed future cost estimates. Actual costs depend on gas prices, RPC fee quotes, and the number of transactions. The scripts check balances, but a positive balance alone does not guarantee enough gas for every later step.

> If a faucet says it succeeded, check the transaction recipient and native balance. Our earlier faucet transaction transferred 25 test LINK; that did not fund BSC gas. A later native BNB claim solved it.

Check local settings and key safely

Keep these non-secret settings in .env. Retain the HYP_KEY value you entered locally; never copy it into this guide.

```
GLD_SUPPLY=1000000
BSC_TESTNET_RPC=https://bsc-testnet.publicnode.com
SEPOLIA_RPC=https://ethereum-sepolia-rpc.publicnode.com
```

Check filenames and restrict access

```
Get-ChildItem -Force .env*
$Identity = [Security.Principal.WindowsIdentity]::GetCurrent().Name
icacls.exe .env /inheritance:r /grant:r "${Identity}:(F)"
```

This removes inherited permissions from the newly created file and grants your current Windows identity full control. It is a local NTFS permission setting; it does not encrypt the key. Keep the project outside shared or automatically synchronized folders.

Print only the signer address

```
$Check = @'
import { Wallet, getAddress } from 'ethers';
try {
  const wallet = new Wallet(process.env.HYP_KEY);
  const owner = getAddress(process.env.OWNER_ADDRESS);
  if (wallet.address !== owner) throw new Error('mismatch');
  console.log('Signer matches owner: ' + wallet.address);
} catch {
  console.error('Invalid key or owner mismatch; key withheld.');
  process.exitCode = 1;
}
'@
$Check | node --env-file=.env --input-type=module
if ($LASTEXITCODE -ne 0) { throw 'Fix .env before continuing.' }
```

Expected: Signer matches owner followed by your public address. If validation fails, re-export the private key for the correct wallet account and edit .env. Do not print HYP_KEY to troubleshoot.

Configure a FRESH route only

Skip this page for existing-route. Its deployment record prevents resetting the deployed route. Use fresh-route only when you deliberately want new contracts and a new 1,000,000 GLD supply.

After editing .env, run the configuration helper. It validates OWNER_ADDRESS, refuses an existing token deployment record, assigns both route owners, sets the BSC collateral placeholder, and starts Sepolia with zero synthetic supply.

```
npm.cmd run configure
if ($LASTEXITCODE -ne 0) { throw 'Configuration failed.' }
```

Inspect the fresh deployment template

```
$RouteDir = 'registry\deployments\warp_routes\GLD'
$DeployConfig = Join-Path $RouteDir 'bsctestnet-sepolia-deploy.yaml'
Get-Content $DeployConfig
```

```
bsctestnet:
  type: collateral
  token: "0x0000000000000000000000000000000000000000"
  owner: "YOUR_PUBLIC_OWNER_ADDRESS"
sepolia:
  type: synthetic
  owner: "YOUR_PUBLIC_OWNER_ADDRESS"
  initialSupply: "0"
```

This excerpt shows the fields to check. The supplied full file also contains mailboxes, metadata, and the default-ISM setting. Do not replace the full file with only this excerpt.

The BSC token zero address is temporary. deploy:token replaces it with the newly deployed ERC-20 address. The warp wrapper intentionally refuses to deploy while it is still zero. A zero router ISM is different: it delegates verification to the mailbox default ISM.

Local route identifier

```
GLD/bsctestnet-sepolia
```

This guide uses the same local route ID in isolated fresh/existing folders. Never merge their deployment records or assume a new route has the worked-example contract addresses.

Compile and run preflight

Run these before broadcasting anything. Compilation and the static checks do not deploy contracts or move tokens.

```
npm.cmd run compile
if ($LASTEXITCODE -ne 0) { throw 'Compile failed.' }
npm.cmd test
if ($LASTEXITCODE -ne 0) { throw 'Static checks failed.' }
npm.cmd run preflight
if ($LASTEXITCODE -ne 0) { throw 'Preflight failed.' }
```

What each command proves

compile: builds contracts/Gold.sol with solc 0.8.38, optimizer runs 200, and the Paris EVM target. Writes artifacts/Gold.json. No Foundry or Hardhat installation is needed.

test: checks constructor encoding, the absence of a public mint function, the two route legs, their owners, and zero initial synthetic supply. It is a static/compilation check, not an EVM execution or complete bridge test. The lab expects GLD_SUPPLY=1000000.

preflight: checks both RPC chain IDs, mailbox contract code, mailbox domains, default ISM code, and the owner native balance. It is read-only.

Expected output

```
Compiled Gold.sol with optimizer and Paris EVM target.
Checks passed: constructor encoding, no public mint, ...
bsctestnet: chain/domain 97, mailbox verified, ...
sepolia: chain/domain 11155111, mailbox verified, ...
```

Both native balances must be funded for deployment. Stop if the RPC reports a different chain, if mailbox code is missing, or if a command exits with a nonzero status. A successful preflight alone does not prove validator delivery compatibility; the real transfers later do.

Deploy Gold on BSC testnet

> Fresh route only. Existing route users should skip to verification and transfers. Do not delete deployments/gold.json to bypass a duplicate-deployment warning.

```
npm.cmd run deploy:token
if ($LASTEXITCODE -ne 0) { throw 'Inspect token deployment status.' }
```

The script validates the key/owner match, checks both gas balances, compiles, estimates token-deployment gas, and broadcasts only to BSC testnet. It mints all 1,000,000 GLD to the owner address.

As soon as broadcasting succeeds, it writes the transaction hash and token address to deployments/gold.json with status pending. It waits for two confirmations, checks the transaction receipt, then reads name, symbol, decimals, totalSupply, and owner balance. After successful readback it marks the record verified and updates the collateral token address in the route config.

```
Get-Content .\deployments\gold.json
Get-Content $DeployConfig
```

Expected fields

```
"chainId": 97
"supply": "1000000000000000000000000"
"status": "verified"
```

The supply is in smallest units: 1,000,000 multiplied by 10^18. Your new token address will differ from the worked-example address. Record it in your worksheet.

If the command stops unexpectedly

Inspect the saved transactionHash on the BSC testnet explorer. A timeout does not prove a transaction failed. If the record is pending, do not rerun token deployment or remove the record. First reconcile the receipt and the token contract state; do not create an accidental second supply.

If no transaction was broadcast and no deployment record exists, fix the reported input/RPC/gas problem before trying again.

Deploy and connect the route

> Fresh route only. This step deploys contracts on both chains and spends test gas. Keep the same owner account and do not switch to mainnet RPC endpoints.

```
npm.cmd run warp:deploy
```

The wrapper runs Hyperlane 45.0.1 with the local registry and route ID. It checks that the BSC token address matches the verified deployment record. Read the CLI deployment plan and accept its confirmation when it shows the intended two chains, owner, collateral/synthetic types, and mailboxes.

Hyperlane deploys implementations, proxy administration contracts and routers, enrolls the remote router addresses, and configures destination gas. Wait until it prints that deployment is complete and writes artifacts.

Inspect the generated route artifacts

```
Get-ChildItem $RouteDir
$CoreConfig = Join-Path $RouteDir 'bsctestnet-sepolia-config.yaml'
Get-Content $CoreConfig
```

Expect two token entries. The BSC entry is EvmHypCollateral; collateralAddressOrDenom is the original ERC-20, while addressOrDenom is the collateral router. The Sepolia addressOrDenom is both the synthetic token and its router. Save both the -deploy.yaml and -config.yaml files.

Normalize the initializer-only field after deployment

CLI 45.0.1 reported an empty on-chain actual value when comparing initialSupply against "0". After the successful zero-supply initialization, remove that deployment-time expectation before running the checker. Current backing and supply are checked directly in the next step.

```
$Lines = Get-Content $DeployConfig
$Lines = $Lines | Where-Object { $_ -notmatch '^\s*initialSupply:' }
$Lines | Set-Content -LiteralPath $DeployConfig -Encoding ascii
```

Explorer source-verification errors can occur without explorer API keys. Distinguish those from reverted deployment transactions or missing artifacts. Direct on-chain verification still needs to pass.

Verify deployed contracts

Existing-route users start here after installing dependencies and setting .env. For a fresh route, wait for deployment to finish first.

```
npm.cmd run warp:check
if ($LASTEXITCODE -ne 0) { throw 'Resolve configuration violations.' }
npm.cmd run verify
if ($LASTEXITCODE -ne 0) { throw 'On-chain verification failed.' }
```

Expected results

warp:check: No violations found. It compares intended settings with the deployed configuration.

verify: prints a JSON report and writes deployments/verification.json. It checks network IDs, router code, ownership, mailboxes, default-ISM selection, remote enrollment in both directions, token metadata, total supply and balances.

```
Get-Content .\deployments\verification.json
```

Collateral invariant

BSC locked GLD = total outstanding Sepolia synthetic GLD when there is no in-flight message and no unsolicited collateral deposit. Immediately after a new route is deployed, both values should be zero. The BSC token supply remains 1,000,000.

The helper compares both chain snapshots. During an in-flight transfer, those values can temporarily differ: source locking/burning occurs before destination minting/releasing. Wait for delivery and retry verification before treating this as a contract defect.

> A configuration check is not a delivery test. Complete the forward and return transfers to verify the selected chains and their existing validator/security setup can actually deliver your route messages.

Transfer 1 GLD to Sepolia

Use the source owner wallet that holds BSC GLD. The CLI defaults the EVM recipient to the signer, so this first test sends to the same public address on Sepolia.

Send with PowerShell argument splatting

```
$SendArgs = @(
  '--origin', 'bsctestnet',
  '--destination', 'sepolia',
  '--amount', '1000000000000000000',
  '--relay', '--timeout', '180'
)
npm.cmd run warp:send -- @SendArgs
```

The amount is in smallest units: 10^18 = 1 GLD. The CLI may first send an ERC-20 approval for the collateral router, then the transfer transaction. Review the prompts and confirm the intended operation.

Save the origin dispatch transaction hash, Hyperlane message ID, and any destination delivery transaction hash. --relay requests CLI self-relaying; it does not replace validators or bypass the destination ISM.

Expected state AFTER delivery

```
BSC owner GLD:           999999
BSC locked GLD:               1
Sepolia owner GLD:            1
Sepolia synthetic supply:     1

npm.cmd run verify
```

> Do not send the return transaction until the Sepolia token balance has arrived. An approval, a successful BSC receipt, or a printed message ID alone does not prove the destination received the tokens.

Return 1 GLD to BSC

Now reverse the chain names. The signer holds the synthetic GLD on Sepolia, which will burn on the source chain. BSC releases the corresponding locked tokens after message verification.

```
$ReturnArgs = @(
  '--origin', 'sepolia',
  '--destination', 'bsctestnet',
  '--amount', '1000000000000000000',
  '--relay', '--timeout', '180'
)
npm.cmd run warp:send -- @ReturnArgs
```

Check final balances and backing

```
npm.cmd run verify
npm.cmd run warp:check

BSC owner GLD:          1000000
BSC locked GLD:               0
Sepolia owner GLD:            0
Sepolia synthetic supply:     0
```

Native test BNB and Sepolia ETH decrease because of gas. The original 1,000,000 GLD supply is unchanged. Save the final verification.json report and the two message IDs as evidence of the round trip.

What if --relay reports metadata is not ready?

An origin burn can succeed even when the immediate self-relay attempt fails. That message remains pending. Wait for validator data, check its delivery status, and retry delivery of that same dispatch transaction using the next page. Do not send another transfer to compensate.

> In the real worked example, the outbound transfer was self-relayed. The immediate return self-relay lacked a validator checkpoint, but another relayer then delivered the existing message successfully. Both final delivery flags were verified on chain.

Check or retry a pending message

Use the existing dispatch transaction hash from the source chain. Replace the placeholder with its complete 0x hash. Use the origin chain where that transaction was sent.

```
$DispatchTx = 'PASTE_EXISTING_DISPATCH_TRANSACTION_HASH'
$StatusArgs = @(
  '--origin', 'sepolia',
  '--dispatchTx', $DispatchTx
)
npm.cmd run status -- @StatusArgs
```

If the message is already delivered, stop. The CLI can show its destination transaction. If it is not delivered, wait for the validator checkpoint to be available, then request self-relaying for the same transaction:

```
npm.cmd run status -- @StatusArgs --relay
```

For a BSC-origin message, change the --origin value in $StatusArgs to bsctestnet. Do not change dispatchTx to an approval hash or a destination delivery hash.

After delivery

```
npm.cmd run verify
```

Meaning of common messages

pending validator / unable to build metadata: the necessary signed checkpoint might not yet be available. Do not weaken the ISM or transfer ownership to work around it.

Merkle proofs are not yet supported: our CLI logged this for one aggregation submodule. Another supported submodule can still satisfy that aggregation if its required signatures are ready. A warning alone is not proof of failed delivery.

origin receipt succeeded: confirms only source execution. A destination mailbox delivery flag or delivered receipt is needed to establish completion.

> A long-lived checkpoint or validator outage is an external dependency. Keep the dispatch hash and message ID, check the explorer and operator status, and retry the existing message when the dependency recovers.

Troubleshooting on Windows

Node/npm is not recognized

Close and reopen PowerShell after installing Node. Run where.exe node and where.exe npm. If neither is found, repair the official Node installation and PATH.

npm.ps1 cannot be loaded

```
npm.cmd --version
npm.cmd ci
```

Use npm.cmd instead of npm. No global execution-policy change is needed.

HYP_KEY invalid / owner mismatch

Check the file is .env, not .env.txt. Use a single account private key, not recovery words. OWNER_ADDRESS must match the funded signing account. Re-run the safe check in step 06; never echo the key.

RPC timeout, rate limit or wrong chain

Update only the relevant RPC URL in .env, save, and rerun preflight. Do not substitute BSC mainnet or Ethereum mainnet endpoints.

```
notepad.exe .env
npm.cmd run preflight
```

Existing deployment record found

Use the deployed route. For a deliberately separate new deployment, extract a separate fresh-route folder. Do not delete the existing record as a routine fix.

Insufficient funds or fee errors

Check native test BNB and Sepolia ETH, not token balances. Fund the correct address on the correct chain. A pending transaction may still consume or reserve gas; inspect the transaction before retrying.

spawn error, ENOENT, or .bin launcher fails

The Windows kit launches the CLI bundle through process.execPath. Use the supplied scripts/warp.mjs, not the older Unix .bin wrapper. Run npm.cmd ci again only when installation is incomplete.

Explorer source verification fails

No API keys were provided in the example. Check receipts and deployed configuration separately. Source publication is incomplete until explorer verification actually succeeds.

Back up and resume safely

Keep the route artifacts and deployment records. They tell the CLI which deployed contracts belong to your route. A new PowerShell session does not require redeployment.

Resume on Windows

```
Set-Location (Join-Path $env:USERPROFILE 'gld-hyperlane-windows')
Set-Location .\GLD-Windows-Kit\existing-route
npm.cmd run preflight
npm.cmd run warp:check
npm.cmd run verify
```

If you used fresh-route, change the last folder name accordingly. Recreate .env locally from your secure source when moving to another machine; it is intentionally absent from the shared ZIP.

Make a public project backup without secrets

```
$Backup = Join-Path $env:USERPROFILE 'gld-route-public-backup'
if (Test-Path $Backup) { throw 'Choose a new backup folder.' }
New-Item -ItemType Directory -Path $Backup | Out-Null
$Items = @(
  'contracts', 'scripts', 'registry', 'deployments',
  'package.json', 'package-lock.json', '.env.example', '.gitignore'
)
foreach ($Item in $Items) {
  if (Test-Path $Item) {
    Copy-Item $Item -Destination $Backup -Recurse
  }
}
Compress-Archive -Path "$Backup\*" -DestinationPath "$Backup.zip"
```

The explicit allowlist excludes .env and node_modules. Inspect backup contents before sharing. Back up the private key separately using your own secure wallet/secret storage; do not add it to this archive.

Windows validation scope

The companion wrappers are adapted to avoid Unix executable paths, and their Node syntax and core checks are validated in the authoring environment. A Windows machine was not available for end-to-end execution. If a Windows dependency install fails, retain the exact non-secret error and consult the pinned package requirements rather than changing chain/security settings.

Our completed deployment

The following addresses belong to the route deployed for this conversation. Fresh deployments will have different addresses. The full public evidence is included in existing-route/deployments/round-trip.json.

Owner

```
0x3ae38EDD6eDfD90180EC897cbF0C5BA527459C83
```

Contracts to import in a wallet

```
BSC testnet Gold ERC-20:
0x8A0D43b12185587e0561691664728bC3030176A9

BSC testnet collateral router (NOT the wallet token):
0xf6285188b3EEb57738f80A4ECeF4bABD5DeDd6b6

Sepolia synthetic GLD token and router:
0x4E2C344358Cd11FAf2CD6454E8c9999D9a45b0E8
```

Import the BSC ERC-20 address on chain 97 and the Sepolia synthetic address on chain 11155111. Both use symbol GLD and 18 decimals. Importing a token changes only wallet display; it does not bridge or mint tokens.

Verified test message IDs

```
BSC to Sepolia:
0x0da974d16bb9832e2b1893cd4d5a42a5231b3b592a645ba7968696aa21686d01

Sepolia to BSC:
0x2865aa0359bfe4d5995e4c5a9c0399a93510a93679b113824478d898ba79f847
```

Look up either message at https://explorer.hyperlane.xyz/message/ followed by the message ID. Direct RPC checks confirmed both source and destination receipts and both mailbox delivery flags. The final checked BSC wallet balance was 1,000,000 GLD; locked collateral and outstanding Sepolia supply were zero.

References and file map

Official sources were checked while preparing the guide. Source requirements and faucet availability may change; the dependency lockfile and the installed CLI help are the authority for this pinned lab.

[1] Node.js downloads: nodejs.org/en/download

[2] Microsoft WinGet install reference: learn.microsoft.com/.../winget/install

[3] BNB faucet documentation: docs.bnbchain.org/docs/bsc-faucet/

[4] GHOST BNB testnet faucet: ghostchain.io/faucet/bnb-testnet/

[5] Hyperlane CLI reference: docs.hyperlane.xyz/.../developer-tools/cli

[6] Hyperlane canonical registry: github.com/hyperlane-xyz/hyperlane-registry

[7] Official EVM Warp Route guide source: hyperlane-xyz/v3-docs: deploy-warp-route.mdx

Project files

contracts/Gold.sol defines the fixed-supply token. scripts/gld.mjs compiles, preflights and deploys it. scripts/configure.mjs prepares a fresh owner/config. scripts/warp.mjs invokes deploy/check/send. scripts/status.mjs checks or retries an existing message. scripts/verify.mjs reads on-chain state and backing. registry contains the official chain snapshots and route artifacts. deployments stores public records.

Complete source appendix

The following pages reproduce the fresh kit configuration and helper source for reference. Long source lines may visually wrap; use the actual files in the ZIP when copying code. All executable PowerShell command blocks in the main guide are supplied without hidden prompts or ellipses. No private keys are included.

> The separate Nine-Beginner-Practicals.pdf contains nine short lab sheets that reuse this project. They do not require additional chains, new tooling, or mainnet funds.

APPENDIX 01 / COMPLETE SOURCE

package.json

Reference listing; the companion ZIP contains the exact, executable file.

```
{
  "name": "gld-testnet-warp-route",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "compile": "node scripts/gld.mjs compile",
    "preflight": "node --env-file-if-exists=.env scripts/gld.mjs preflight",
    "deploy:token": "node --env-file-if-exists=.env scripts/gld.mjs deploy",
    "test": "node --env-file-if-exists=.env scripts/gld.mjs test",
    "hyperlane": "hyperlane",
    "warp:deploy": "node --env-file-if-exists=.env scripts/warp.mjs deploy",
    "warp:check": "node --env-file-if-exists=.env scripts/warp.mjs check",
    "warp:send": "node --env-file-if-exists=.env scripts/warp.mjs send",
    "verify": "node --env-file-if-exists=.env scripts/verify.mjs",
    "configure": "node --env-file=.env scripts/configure.mjs",
    "status": "node --env-file=.env scripts/status.mjs"
  },
  "dependencies": {
    "@hyperlane-xyz/cli": "45.0.1",
    "@openzeppelin/contracts": "5.6.1",
    "ethers": "6.17.0",
    "solc": "0.8.38",
    "yaml": "2.9.1"
  }
}
```

APPENDIX 02 / COMPLETE SOURCE

.env.example

Reference listing; the companion ZIP contains the exact, executable file.

```
# Use a dedicated TESTNET wallet. Never commit .env or paste its key in chat.
HYP_KEY=
OWNER_ADDRESS=0x3ae38EDD6eDfD90180EC897cbF0C5BA527459C83
GLD_SUPPLY=1000000
BSC_TESTNET_RPC=https://bsc-testnet.publicnode.com
SEPOLIA_RPC=https://ethereum-sepolia-rpc.publicnode.com
```

APPENDIX 03 / COMPLETE SOURCE

contracts/Gold.sol

Reference listing; the companion ZIP contains the exact, executable file.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

/// @notice Fixed-supply test token. Does not represent or redeem physical gold.
contract Gold is ERC20 {
    constructor(address recipient, uint256 supply) ERC20("Gold", "GLD") {
        require(recipient != address(0), "Zero recipient");
        _mint(recipient, supply);
    }
}
```

APPENDIX 04 / COMPLETE SOURCE

scripts/configure.mjs

Reference listing; the companion ZIP contains the exact, executable file.

```
import assert from 'node:assert/strict';
import { existsSync, readFileSync, writeFileSync } from 'node:fs';
import { getAddress, ZeroAddress } from 'ethers';
import { parse, stringify } from 'yaml';
assert(!existsSync('deployments/gold.json'),
  'Existing deployment found. Use it; do not reset this folder.');
assert(process.env.OWNER_ADDRESS, 'Set OWNER_ADDRESS in .env first.');
const owner = getAddress(process.env.OWNER_ADDRESS);
assert.notEqual(owner, ZeroAddress);
const path = 'registry/deployments/warp_routes/GLD/' +
  'bsctestnet-sepolia-deploy.yaml';
const config = parse(readFileSync(path, 'utf8'));
for (const leg of Object.values(config)) leg.owner = owner;
config.bsctestnet.token = ZeroAddress;
config.sepolia.initialSupply = '0';
writeFileSync(path, stringify(config));
console.log('Configured fresh route for public owner: ' + owner);
```

APPENDIX 05 / COMPLETE SOURCE

scripts/gld.mjs

Reference listing; the companion ZIP contains the exact, executable file.

```
import assert from 'node:assert/strict';
import { existsSync, mkdirSync, readFileSync, writeFileSync } from 'node:fs';
import { resolve } from 'node:path';
import solc from 'solc';
import { parse as parseYaml } from 'yaml';
import { Contract, ContractFactory, FetchRequest, JsonRpcProvider, Wallet, ZeroAddress, formatEther,
    getAddress, parseUnits } from 'ethers';

const OWNER = '0x3ae38EDD6eDfD90180EC897cbF0C5BA527459C83';
const routeFile = 'registry/deployments/warp_routes/GLD/bsctestnet-sepolia-deploy.yaml';
const deploymentFile = 'deployments/gold.json';
const chains = {
  bsctestnet: { id: 97, rpc: process.env.BSC_TESTNET_RPC || 'https://bsc-testnet.publicnode.com' },
  sepolia: { id: 11155111, rpc: process.env.SEPOLIA_RPC || 'https://ethereum-sepolia-rpc.publicnode.com'
    },
};

function save(path, value) {
  mkdirSync(resolve(path, '..'), { recursive: true });
  writeFileSync(path, JSON.stringify(value, null, 2) + '\n');
}

function settings() {
  const owner = getAddress(process.env.OWNER_ADDRESS || OWNER);
  assert.notEqual(owner, ZeroAddress, 'Owner must be nonzero');
  const supply = parseUnits(process.env.GLD_SUPPLY || '1000000', 18);
  assert(supply > 0n, 'Supply must be positive');
  return { owner, supply };
}

function compile() {
  const input = {
    language: 'Solidity',
    sources: { 'Gold.sol': { content: readFileSync('contracts/Gold.sol', 'utf8') } },
```

```
    settings: {
      optimizer: { enabled: true, runs: 200 },
      evmVersion: 'paris',
      outputSelection: { '*': { '*': ['abi', 'evm.bytecode.object', 'evm.deployedBytecode.object'] } },
    },
  };
  const output = JSON.parse(solc.compile(JSON.stringify(input), {
    import: path => {
      if (!path.startsWith('@openzeppelin/contracts/')) return { error: 'Unexpected import' };
      return { contents: readFileSync(resolve('node_modules', path), 'utf8') };
    },
  }));
  const errors = (output.errors || []).filter(e => e.severity === 'error');
  assert.equal(errors.length, 0, errors.map(e => e.formattedMessage).join('\n'));
  const contract = output.contracts['Gold.sol'].Gold;
  const artifact = { compiler: solc.version(), abi: contract.abi, bytecode: '0x' +
    contract.evm.bytecode.object };
  save('artifacts/Gold.json', artifact);
  console.log('Compiled Gold.sol with optimizer and Paris EVM target.');
  return artifact;
}

async function inspect(name, chain, owner) {
  const request = new FetchRequest(chain.rpc);
  request.timeout = 20000;
  const provider = new JsonRpcProvider(request);
  try {
    const network = await provider.getNetwork();
    assert.equal(network.chainId, BigInt(chain.id), `${name}: wrong chain`);
    const addresses = parseYaml(readFileSync(`registry/chains/${name}/addresses.yaml`, 'utf8'));
    assert.notEqual(await provider.getCode(addresses.mailbox), '0x', `${name}: missing mailbox code`);
    const mailbox = new Contract(addresses.mailbox, ['function localDomain() view returns (uint32)',
    'function defaultIsm() view returns (address)'], provider);
    assert.equal(await mailbox.localDomain(), BigInt(chain.id), `${name}: wrong mailbox domain`);

    const ism = await mailbox.defaultIsm();
    assert.notEqual(ism, ZeroAddress, `${name}: zero default ISM`);
    assert.notEqual(await provider.getCode(ism), '0x', `${name}: missing ISM code`);
    const balance = await provider.getBalance(owner);
    console.log(`${name}: chain/domain ${chain.id}, mailbox verified, default ISM ${ism}, owner gas
    balance ${formatEther(balance)}`);
    return { name, balance };
  } finally {
    provider.destroy();
  }
}

async function preflight() {
  const { owner } = settings();
  console.log(`Owner and supply recipient: ${owner}`);
  const results = await Promise.allSettled(Object.entries(chains).map(([name, chain]) => inspect(name,
    chain, owner)));
  const failures = results.filter(r => r.status === 'rejected');
  for (const failure of failures) console.error(failure.reason.shortMessage || failure.reason.message);
  assert.equal(failures.length, 0, 'RPC preflight failed; update RPC URLs in .env and retry');
  return results.map(r => r.value);
}

async function deploy() {
  assert(!existsSync(deploymentFile), 'Existing deployment record found; reuse it instead of deploying
    another supply.');
  assert(process.env.HYP_KEY, 'Set HYP_KEY locally in .env; do not send it in chat.');
  const { owner, supply } = settings();
  const wallet = new Wallet(process.env.HYP_KEY);
  assert.equal(wallet.address, owner, 'Use the owner wallet for deployment and subsequent bridge
    tests.');
  const checks = await preflight();
  assert(checks.every(c => c.balance > 0n), 'Fund owner with test BNB and Sepolia ETH first.');
  const artifact = compile();
  const request = new FetchRequest(chains.bsctestnet.rpc);
  request.timeout = 20000;
```

```
  const provider = new JsonRpcProvider(request);
  try {
    const factory = new ContractFactory(artifact.abi, artifact.bytecode, wallet.connect(provider));
    const tx = await factory.getDeployTransaction(owner, supply);
    const gas = await provider.estimateGas({ ...tx, from: owner });
    const fees = await provider.getFeeData();
    const gasPrice = fees.maxFeePerGas || fees.gasPrice;
    assert(gasPrice, 'RPC did not provide gas prices');
    assert(await provider.getBalance(owner) > gas * gasPrice * 12n / 10n, 'Insufficient BNB for estimated
     token deployment cost');
    console.log(`Estimated token deployment gas: ${gas}. Broadcasting to BSC TESTNET only.`);
    const token = await factory.deploy(owner, supply);
    // Persist immediately: a timeout must not silently lead to a second supply.
    const record = { chain: 'bsctestnet', chainId: 97, token: await token.getAddress(), owner, supply:
    supply.toString(), transactionHash: token.deploymentTransaction().hash, status: 'pending' };
    save(deploymentFile, record);
    const receipt = await token.deploymentTransaction().wait(2);
    assert.equal(receipt.status, 1, 'Token deployment reverted');
    assert.equal(await token.name(), 'Gold');
    assert.equal(await token.symbol(), 'GLD');
    assert.equal(await token.decimals(), 18n);
    assert.equal(await token.totalSupply(), supply);
    assert.equal(await token.balanceOf(owner), supply);
    record.status = 'verified';
    save(deploymentFile, record);
    const config = parseYaml(readFileSync(routeFile, 'utf8'));
    config.bsctestnet.token = record.token;
    for (const leg of Object.values(config)) leg.owner = owner;
    save(routeFile, config);
    console.log(`Gold deployed and verified: ${record.token}. Warp config updated.`);
  } finally {
    provider.destroy();
  }
}

async function test() {
  const artifact = compile();
  const { owner, supply } = settings();
  assert.equal(supply, 1000000n * 10n ** 18n);
  const factory = new ContractFactory(artifact.abi, artifact.bytecode);
  const tx = await factory.getDeployTransaction(owner, supply);
  assert(tx.data.startsWith(artifact.bytecode));
  assert(!artifact.abi.some(item => item.type === 'function' && item.name === 'mint'), 'Fixed-supply
    token must not expose mint');
  const config = parseYaml(readFileSync(routeFile, 'utf8'));
  assert.deepEqual(Object.keys(config).sort(), ['bsctestnet', 'sepolia']);
  assert.equal(config.bsctestnet.type, 'collateral');
  assert.equal(config.sepolia.type, 'synthetic');
  assert.equal(config.sepolia.initialSupply ?? '0', '0');
  assert.equal(config.bsctestnet.owner, owner);
  assert.equal(config.sepolia.owner, owner);
  console.log('Checks passed: constructor encoding, no public mint, route chains/types/owners, zero
    initial synthetic supply.');
}

try {
  const command = process.argv[2];
  if (command === 'compile') compile();
  else if (command === 'test') await test();
  else if (command === 'preflight') await preflight();
  else if (command === 'deploy') await deploy();
  else throw new Error('Use compile, test, preflight, or deploy');
} catch (error) {
  // Provider error objects can include raw signed transactions; print only a summary.
  console.error(error.shortMessage || error.message);
  process.exitCode = 1;
}
```

APPENDIX 06 / COMPLETE SOURCE

scripts/warp.mjs

Reference listing; the companion ZIP contains the exact, executable file.

```
import assert from 'node:assert/strict';
import { readFileSync, writeFileSync } from 'node:fs';
import { spawnSync } from 'node:child_process';
import { ZeroAddress, getAddress } from 'ethers';
import { parse as parseYaml } from 'yaml';

const action = process.argv[2];
assert(['deploy', 'check', 'send'].includes(action), 'Use deploy, check, or send');
const config = parseYaml(readFileSync('registry/deployments/warp_routes/GLD/bsctestnet-sepolia-
    deploy.yaml', 'utf8'));
assert.notEqual(config.bsctestnet.token, ZeroAddress, 'Run npm run deploy:token first; collateral token
    is still a placeholder.');
const record = JSON.parse(readFileSync('deployments/gold.json', 'utf8'));
assert.equal(record.status, 'verified', 'Verify the pending token deployment before deploying the
    route.');
assert.equal(getAddress(config.bsctestnet.token), getAddress(record.token), 'Collateral address differs
    from deployment record');
// Keep Hyperlane and the token deployer on the same RPC endpoints.
for (const [chain, variable] of [['bsctestnet', 'BSC_TESTNET_RPC'], ['sepolia', 'SEPOLIA_RPC']]) {
  if (process.env[variable]) {
    const path = `registry/chains/${chain}/metadata.yaml`;
    const metadata = parseYaml(readFileSync(path, 'utf8'));
    metadata.rpcUrls = [{ http: process.env[variable] }];
    writeFileSync(path, JSON.stringify(metadata, null, 2) + '\n');
  }
}
const result = spawnSync(process.execPath, [
  'node_modules/@hyperlane-xyz/cli/bundle/index.js',
  'warp', action, '--registry', './registry', '--warp-route-id', 'GLD/bsctestnet-sepolia',
    ...process.argv.slice(3),
], { stdio: 'inherit', env: process.env });
if (result.error) throw result.error;
process.exitCode = result.status ?? 1;
```

APPENDIX 07 / COMPLETE SOURCE

scripts/status.mjs

Reference listing; the companion ZIP contains the exact, executable file.

```
import { spawnSync } from 'node:child_process';
const result = spawnSync(process.execPath, [
  'node_modules/@hyperlane-xyz/cli/bundle/index.js',
  'status', '--registry', './registry', ...process.argv.slice(2),
], { stdio: 'inherit', env: process.env });
if (result.error) throw result.error;
process.exitCode = result.status ?? 1;
```

scripts/verify.mjs

Reference listing; the companion ZIP contains the exact, executable file.

```
import assert from 'node:assert/strict';
import { readFileSync, writeFileSync } from 'node:fs';
import { parse } from 'yaml';
import { Contract, FetchRequest, JsonRpcProvider, ZeroAddress, formatEther, formatUnits, getAddress,
    zeroPadValue } from 'ethers';

const record = JSON.parse(readFileSync('deployments/gold.json', 'utf8'));
const route = parse(readFileSync('registry/deployments/warp_routes/GLD/bsctestnet-sepolia-config.yaml',
    'utf8'));
const deploy = parse(readFileSync('registry/deployments/warp_routes/GLD/bsctestnet-sepolia-deploy.yaml',
    'utf8'));
const abi = ['function name() view returns (string)', 'function symbol() view returns (string)',
    'function decimals() view returns (uint8)', 'function totalSupply() view returns (uint256)',
    'function balanceOf(address) view returns (uint256)'];
const routerAbi = [...abi, 'function owner() view returns (address)', 'function mailbox() view returns
    (address)', 'function interchainSecurityModule() view returns (address)', 'function routers(uint32)
    view returns (bytes32)', 'function wrappedToken() view returns (address)'];
const providers = [];

try {
  const results = await Promise.all(route.tokens.map(async leg => {
    const metadata = parse(readFileSync(`registry/chains/${leg.chainName}/metadata.yaml`, 'utf8'));
    const request = new FetchRequest(process.env[leg.chainName === 'bsctestnet' ? 'BSC_TESTNET_RPC' :
    'SEPOLIA_RPC'] || metadata.rpcUrls[0].http);
    request.timeout = 20000;
    const provider = new JsonRpcProvider(request);
    providers.push(provider);
    assert.equal((await provider.getNetwork()).chainId, BigInt(metadata.chainId));
    assert.notEqual(await provider.getCode(leg.addressOrDenom), '0x');
    const router = new Contract(leg.addressOrDenom, routerAbi, provider);
    assert.equal(await router.owner(), getAddress(record.owner));
    assert.equal(await router.mailbox(), getAddress(deploy[leg.chainName].mailbox));
    assert.equal(await router.interchainSecurityModule(), ZeroAddress);
    const remote = route.tokens.find(token => token.chainName !== leg.chainName);
    const remoteMetadata = parse(readFileSync(`registry/chains/${remote.chainName}/metadata.yaml`,
    'utf8'));
    assert.equal((await router.routers(remoteMetadata.domainId)).toLowerCase(),
    zeroPadValue(remote.addressOrDenom, 32).toLowerCase());
    const tokenAddress = leg.collateralAddressOrDenom || leg.addressOrDenom;
    const token = new Contract(tokenAddress, abi, provider);
    assert.equal(await token.name(), 'Gold');
    assert.equal(await token.symbol(), 'GLD');
```

```
    assert.equal(await token.decimals(), 18n);
    const [balance, totalSupply, nativeBalance] = await Promise.all([token.balanceOf(record.owner),
    token.totalSupply(), provider.getBalance(record.owner)]);
    const result = { chain: leg.chainName, chainId: metadata.chainId, router: leg.addressOrDenom, token:
    tokenAddress, owner: record.owner, ownerGLD: formatUnits(balance, 18), totalSupplyGLD:
    formatUnits(totalSupply, 18), ownerNativeBalance: formatEther(nativeBalance), remoteRouterVerified:
    true };
    if (leg.chainName === 'bsctestnet') {
      assert.equal(await router.wrappedToken(), getAddress(record.token));
      assert.equal(totalSupply, BigInt(record.supply));
      result.lockedGLD = formatUnits(await token.balanceOf(leg.addressOrDenom), 18);
    }
    return result;
  }));
  const bsc = results.find(result => result.chain === 'bsctestnet');
  const sepolia = results.find(result => result.chain === 'sepolia');
  assert.equal(bsc.lockedGLD, sepolia.totalSupplyGLD, 'Locked collateral must equal outstanding synthetic
     supply');
  const report = { verifiedAt: new Date().toISOString(), backingInvariantVerified: true, chains: results
    };
  writeFileSync('deployments/verification.json', JSON.stringify(report, null, 2) + '\n');
  console.log(JSON.stringify(report, null, 2));
} catch (error) {
  console.error(error.shortMessage || error.message);
  process.exitCode = 1;
} finally {
  for (const provider of providers) provider.destroy();
}
```

APPENDIX 09 / COMPLETE SOURCE

registry/deployments/warp_routes/GLD/bsctestnet-sepolia-deploy.yaml

Reference listing; the companion ZIP contains the exact, executable file.

```
{
  "bsctestnet": {
    "type": "collateral",
    "token": "0x0000000000000000000000000000000000000000",
    "owner": "0x3ae38EDD6eDfD90180EC897cbF0C5BA527459C83",
    "mailbox": "0xF9F6F5646F478d5ab4e20B0F910C92F1CCC9Cc6D",
    "interchainSecurityModule": "0x0000000000000000000000000000000000000000",
    "name": "Gold",
    "symbol": "GLD",
    "decimals": 18
  },
  "sepolia": {
    "type": "synthetic",
    "owner": "0x3ae38EDD6eDfD90180EC897cbF0C5BA527459C83",
    "mailbox": "0xfFAEF09B3cd11D9b20d1a19bECca54EEC2884766",
    "interchainSecurityModule": "0x0000000000000000000000000000000000000000",
    "name": "Gold",
    "symbol": "GLD",
    "decimals": 18,
    "initialSupply": "0"
  }
}
```